

✓ VisionCare AI — Fixed & Balanced Training

Run each cell in order | Enable GPU: Runtime → Change runtime type → T4 GPU

✓ Step 1 — Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
print('Drive mounted!')
```

```
Mounted at /content/drive
Drive mounted!
```

✓ Step 2 — Import Libraries

```
import os, random, shutil, warnings
import numpy as np
import tensorflow as tf
from pathlib import Path
from PIL import Image as PILImage
from collections import Counter
import matplotlib.pyplot as plt
import seaborn as sns
```

```
warnings.filterwarnings('ignore')
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

```
print('TensorFlow:', tf.__version__)
print('GPU:', tf.config.list_physical_devices('GPU'))
```

```
TensorFlow: 2.20.0
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

✓ Step 3 — Configuration

```
RAW_DATA    = Path('/content/drive/MyDrive/eyes_data_project/data-2/Augr')
WORK_DIR    = Path('/content/eye_balanced')
OUTPUT_DIR  = Path('/content/visioncare_outputs')
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

```
IMG_SIZE    = 224
```

```
BATCH_SIZE      = 32
TARGET_PER_CLASS = 2500
```

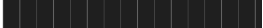
```
print('Dataset path :', RAW_DATA)
print('Target/class :', TARGET_PER_CLASS, 'images')
```

```
Dataset path : /content/drive/MyDrive/eyes_data_project/data-2/Augmented
Target/class : 2500 images
```

✓ Step 4 — Check Original Class Distribution

```
print('Original class distribution:')
print('-' * 55)
total = 0
for cls in sorted(os.listdir(RAW_DATA)):
    if cls.startswith('.'): continue
    n = len(list((RAW_DATA / cls).iterdir()))
    total += n
    bar = chr(9608) * (n // 100)
    print('  {:<45} {:>5} {}'.format(cls, n, bar))
print('-' * 55)
print('  Total:', total)
```

Original class distribution:

Central Serous Chorioretinopathy	606	
Diabetic Retinopathy	3444	
Disc Edema	762	
Glaucoma	2880	
Healthy	2676	
Macular Scar	1937	
Myopia	2251	
Pterygium	102	
Retinal Detachment	750	
Retinitis Pigmentosa	834	

Total:	16242	

✓ Step 5 — Balance Dataset

Oversample minority classes so every class has **2500 images**. This fixes the model predicting only one class.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, ...

aug = ImageDataGenerator(
    rotation_range      = 40,
    width_shift_range   = 0.2,
    height_shift_range  = 0.2,
```

```

horizontal_flip    = True,
vertical_flip      = True,
zoom_range         = 0.25,
brightness_range   = [0.6, 1.4],
shear_range        = 0.15,
fill_mode          = 'nearest',
rescale            = None
)

for cls_dir in sorted(RAW_DATA.iterdir()):
    if not cls_dir.is_dir() or cls_dir.name.startswith('.'): continue
    cls_name = cls_dir.name
    out_cls = WORK_DIR / cls_name
    out_cls.mkdir(parents=True, exist_ok=True)
    images = list(cls_dir.glob('*.jpg')) + list(cls_dir.glob('*.png'))
    print('\n  {:<45} original: {}'.format(cls_name, len(images)))
    for img_path in images:
        dst = out_cls / img_path.name
        if not dst.exists():
            shutil.copy(img_path, dst)
    current = len(list(out_cls.iterdir()))
    needed = TARGET_PER_CLASS - current
    if needed > 0:
        print('    generating {} augmented images...'.format(needed))
        generated = 0
        while generated < needed:
            src = random.choice(images)
            img = load_img(src, target_size=(IMG_SIZE, IMG_SIZE))
            x = np.expand_dims(img_to_array(img), axis=0)
            for batch in aug.flow(x, batch_size=1):
                PILImage.fromarray(batch[0].astype('uint8')).save(out_cls)
                generated += 1
            if generated >= needed: break
        print('    final: {}'.format(len(list(out_cls.iterdir()))))

print('\nBalancing complete!')
```

Central Serous Chorioretinopathy	original: 606
generating 1894 augmented images...	
final: 2500	
Diabetic Retinopathy	original: 3444
final: 3444	
Disc Edema	original: 762
generating 1738 augmented images...	
final: 2500	
Glaucoma	original: 2880
final: 2880	
Healthy	original: 2676
final: 2676	
Macular Scar	original: 1937

```

generating 563 augmented images...
final: 2500

Myopia                                original: 2251
generating 249 augmented images...
final: 2500

Pterygium                             original: 102
generating 2398 augmented images...
final: 2500

Retinal Detachment                    original: 750
generating 1750 augmented images...
final: 2500

Retinitis Pigmentosa                  original: 834
generating 1666 augmented images...
final: 2500

Balancing complete!

```

✓ Step 6 — Verify Balanced Distribution

```

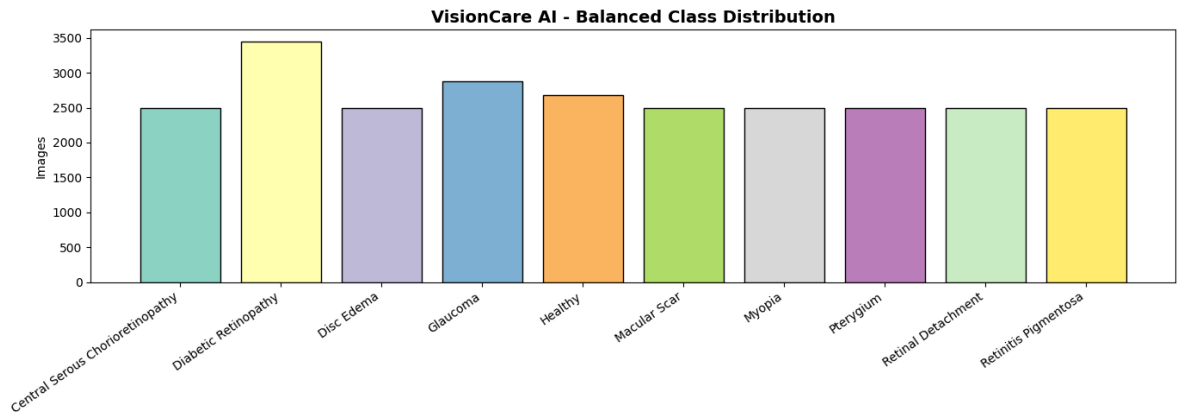
print('Balanced class distribution:')
print('-' * 55)
counts = {}
for cls_dir in sorted(WORK_DIR.iterdir()):
    if cls_dir.is_dir():
        n = len(list(cls_dir.iterdir()))
        counts[cls_dir.name] = n
        bar = chr(9608) * (n // 100)
        print('  {:<45} {:>5} {}'.format(cls_dir.name, n, bar))

fig, ax = plt.subplots(figsize=(14, 5))
ax.bar(counts.keys(), counts.values(),
       color=plt.cm.Set3(np.linspace(0, 1, len(counts))), edgecolor='k')
ax.set_title('VisionCare AI - Balanced Class Distribution', fontsize=14)
ax.set_ylabel('Images')
plt.xticks(rotation=35, ha='right')
plt.tight_layout()
plt.show()

```

Balanced class distribution:

Central Serous Chorioretinopathy	2500	
Diabetic Retinopathy	3444	
Disc Edema	2500	
Glaucoma	2880	
Healthy	2676	
Macular Scar	2500	
Myopia	2500	
Pterygium	2500	
Retinal Detachment	2500	
Retinitis Pigmentosa	2500	



✓ Step 7 — Data Generators

```

train_datagen = ImageDataGenerator(
    rescale      = 1./255,
    validation_split = 0.2,
    rotation_range  = 20,
    horizontal_flip = True,
    zoom_range     = 0.15,
    fill_mode      = 'nearest'
)

val_datagen = ImageDataGenerator(
    rescale      = 1./255,
    validation_split = 0.2
)

train_gen = train_datagen.flow_from_directory(
    WORK_DIR, target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE, class_mode='categorical',
    subset='training', seed=SEED, shuffle=True

```

```

)
val_gen = val_datagen.flow_from_directory(
    WORK_DIR, target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE, class_mode='categorical',
    subset='validation', seed=SEED, shuffle=False
)

NUM_CLASSES = len(train_gen.class_indices)
idx_to_class = {v: k for k, v in train_gen.class_indices.items()}

print('Training samples  :', train_gen.samples)
print('Validation samples:', val_gen.samples)
print('Classes           :', NUM_CLASSES)
print('Class map          :', train_gen.class_indices)

```

```

Found 21201 images belonging to 10 classes.
Found 5299 images belonging to 10 classes.
Training samples  : 21201
Validation samples: 5299
Classes           : 10
Class map          : {'Central Serous Chorioretinopathy': 0, 'Diabetic R

```

✓ Step 8 — Class Weights

```

from sklearn.utils.class_weight import compute_class_weight

labels = train_gen.classes
weights = compute_class_weight('balanced', classes=np.unique(labels),
                               class_weight_dict = dict(enumerate(weights)))

print('Class weights (should all be ~1.0 since data is balanced):')
for i, w in class_weight_dict.items():
    print('  {:<45} {:.3f}'.format(idx_to_class[i], w))

```

```

Class weights (should all be ~1.0 since data is balanced):
Central Serous Chorioretinopathy      1.060
Diabetic Retinopathy                  0.769
Disc Edema                            1.060
Glaucoma                             0.920
Healthy                               0.990
Macular Scar                          1.060
Myopia                                1.060
Pterygium                             1.060
Retinal Detachment                    1.060
Retinitis Pigmentosa                  1.060

```

✓ Step 9 — Build EfficientNetB0 Model

```

from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras import layers, callbacks, optimizers

```

```

base_model = EfficientNetB0(
    include_top = False,
    weights      = 'imagenet',
    input_shape  = (IMG_SIZE, IMG_SIZE, 3)
)
base_model.trainable = False

inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = base_model(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(512, activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.4)(x)
x = layers.Dense(256, activation='relu')(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(NUM_CLASSES, activation='softmax')(x)

model = tf.keras.Model(inputs, outputs, name='VisionCare_EfficientNetB0')
model.summary()

```

Downloading data from <https://storage.googleapis.com/keras-applications/16705208/16705208> **2s** 0us/step

Model: "VisionCare EfficientNetB0"

Layer (type)	Output Shape	Param
input_layer_1 (InputLayer)	(None , 224 , 224 , 3)	
efficientnetb0 (Functional)	(None , 7 , 7 , 1280)	4,049
global_average_pooling2d (GlobalAveragePooling2D)	(None , 1280)	
dense (Dense)	(None , 512)	655
batch_normalization (BatchNormalization)	(None , 512)	2
dropout (Dropout)	(None , 512)	
dense_1 (Dense)	(None , 256)	131
dropout_1 (Dropout)	(None , 256)	
dense_2 (Dense)	(None , 10)	2

Total params: [4,841,389](#) (18.47 MB)

Trainable params: [790,794](#) (3.02 MB)

Non-trainable params: [4,050,595](#) (15.45 MB)

✓ 🚀 Step 10 — Phase 1: Train Head Only

Base is **frozen**. Only classifier layers train. LR = [0.001](#)

```

model.compile(
    optimizer = optimizers.Adam(0.001),
    loss      = 'categorical_crossentropy',
    metrics   = ['accuracy']
)

cb1 = [
    callbacks.EarlyStopping(monitor='val_accuracy', patience=6,
                           restore_best_weights=True, verbose=1),
    callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5,
                                patience=3, min_lr=0.000001, verbose=1),
    callbacks.ModelCheckpoint(str(OUTPUT_DIR / 'phase1_best.keras'),
                              monitor='val_accuracy', save_best_only=True)
]

history1 = model.fit(
    train_gen,
    epochs      = 20,
    validation_data = val_gen,
    class_weight = class_weight_dict,
    callbacks    = cb1
)

print('Phase 1 best val_accuracy:', round(max(history1.history['val_ac

```

```

Epoch 1/20
663/663 ━━━━━━━━━━━ 0s 675ms/step - accuracy: 0.1026 - loss: 2.0000
Epoch 1: val_accuracy improved from None to 0.14569, saving model to /content/visioncare_outputs/phase1_best.keras
Epoch 1: finished saving model to /content/visioncare_outputs/phase1_best.keras
663/663 ━━━━━━━━━━━ 571s 812ms/step - accuracy: 0.1007 - loss: 2.0000
Epoch 2/20
663/663 ━━━━━━━━━━━ 0s 583ms/step - accuracy: 0.1017 - loss: 2.0000
Epoch 2: val_accuracy did not improve from 0.14569
663/663 ━━━━━━━━━━━ 455s 686ms/step - accuracy: 0.1005 - loss: 2.0000
Epoch 3/20
663/663 ━━━━━━━━━━━ 0s 598ms/step - accuracy: 0.1047 - loss: 2.0000
Epoch 3: val_accuracy did not improve from 0.14569
663/663 ━━━━━━━━━━━ 462s 697ms/step - accuracy: 0.1048 - loss: 2.0000
Epoch 4/20
663/663 ━━━━━━━━━━━ 0s 596ms/step - accuracy: 0.1133 - loss: 2.0000
Epoch 4: val_accuracy did not improve from 0.14569
663/663 ━━━━━━━━━━━ 462s 697ms/step - accuracy: 0.1180 - loss: 2.0000
Epoch 5/20
663/663 ━━━━━━━━━━━ 0s 598ms/step - accuracy: 0.1240 - loss: 2.0000
Epoch 5: ReduceLROnPlateau reducing learning rate to 0.0005000000023748
Epoch 5: val_accuracy did not improve from 0.14569
663/663 ━━━━━━━━━━━ 465s 701ms/step - accuracy: 0.1328 - loss: 2.0000
Epoch 6/20
663/663 ━━━━━━━━━━━ 0s 598ms/step - accuracy: 0.1470 - loss: 2.0000
Epoch 6: val_accuracy did not improve from 0.14569
663/663 ━━━━━━━━━━━ 465s 701ms/step - accuracy: 0.1475 - loss: 2.0000
Epoch 7/20
663/663 ━━━━━━━━━━━ 0s 631ms/step - accuracy: 0.1533 - loss: 2.0000
Epoch 7: val_accuracy did not improve from 0.14569
663/663 ━━━━━━━━━━━ 486s 733ms/step - accuracy: 0.1545 - loss: 2.0000
Epoch 7: early stopping
Restoring model weights from the end of the best epoch: 1.

```


Phase 1 best val_accuracy: 14.57 %

- Step 11 — Phase 2: Fine-tune Top 50 Layers

Unfreeze top 50 layers. Very small LR = 0.00001

```
base_model.trainable = True
for layer in base_model.layers[:-50]:
    layer.trainable = False

print('Trainable layers:', sum(1 for l in model.layers if l.trainable))

model.compile(
    optimizer = optimizers.Adam(0.00001),
    loss      = 'categorical_crossentropy',
    metrics   = ['accuracy']
)

cb2 = [
    callbacks.EarlyStopping(monitor='val_accuracy', patience=10,
                             restore_best_weights=True, verbose=1),
    callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.3,
                                 patience=4, min_lr=0.00000001, verbose=1),
    callbacks.ModelCheckpoint(str(OUTPUT_DIR / 'visioncare_efficientnetv2_{}_acc.h5'.format(
        monitor='val_accuracy', save_best_only=True)))
]

history2 = model.fit(
    train_gen,
    epochs          = 50,
    validation_data = val_gen,
    class_weight    = class_weight_dict,
    callbacks       = cb2
)

print('Phase 2 best val_accuracy:', round(max(history2.history['val_ac
```

```
Trainable layers: 9
Epoch 1/50
663/663  0s 638ms/step - accuracy: 0.0984 - loss: 0.4700
Epoch 1: val_accuracy improved from None to 0.09436, saving model to /content/visioncare_outputs/visioncare_model_1.h5
Epoch 1: finished saving model to /content/visioncare_outputs/visioncare_model_1.h5
663/663  541s 762ms/step - accuracy: 0.0988 - loss: 0.4699
Epoch 2/50
663/663  0s 597ms/step - accuracy: 0.1092 - loss: 0.4699
Epoch 2: val_accuracy did not improve from 0.09436
663/663  462s 697ms/step - accuracy: 0.1103 - loss: 0.4699
Epoch 3/50
663/663  0s 598ms/step - accuracy: 0.1134 - loss: 0.4699
Epoch 3: val_accuracy did not improve from 0.09436
663/663  460s 695ms/step - accuracy: 0.1163 - loss: 0.4699
Epoch 4/50
663/663  0s 588ms/step - accuracy: 0.1244 - loss: 0.4699
```

```

Epoch 4: val_accuracy did not improve from 0.09436
663/663 ██████████ 454s 685ms/step - accuracy: 0.1232 - loss: 2.0000
Epoch 5/50
663/663 ██████████ 0s 593ms/step - accuracy: 0.1326 - loss: 2.0000
Epoch 5: ReduceLROnPlateau reducing learning rate to 2.9999999242136253
Epoch 5: val_accuracy did not improve from 0.09436
663/663 ██████████ 462s 697ms/step - accuracy: 0.1292 - loss: 2.0000
Epoch 6/50
663/663 ██████████ 0s 586ms/step - accuracy: 0.1318 - loss: 2.0000
Epoch 6: val_accuracy did not improve from 0.09436
663/663 ██████████ 453s 683ms/step - accuracy: 0.1328 - loss: 2.0000
Epoch 7/50
663/663 ██████████ 0s 610ms/step - accuracy: 0.1370 - loss: 2.0000
Epoch 7: val_accuracy did not improve from 0.09436
663/663 ██████████ 472s 712ms/step - accuracy: 0.1355 - loss: 2.0000
Epoch 8/50
663/663 ██████████ 0s 605ms/step - accuracy: 0.1373 - loss: 2.0000
Epoch 8: val_accuracy did not improve from 0.09436
663/663 ██████████ 471s 711ms/step - accuracy: 0.1392 - loss: 2.0000
Epoch 9/50
663/663 ██████████ 0s 590ms/step - accuracy: 0.1367 - loss: 2.0000
Epoch 9: ReduceLROnPlateau reducing learning rate to 8.9999999636216671e-05
Epoch 9: val_accuracy did not improve from 0.09436
663/663 ██████████ 457s 690ms/step - accuracy: 0.1348 - loss: 2.0000
Epoch 10/50
663/663 ██████████ 0s 590ms/step - accuracy: 0.1398 - loss: 2.0000
Epoch 10: val_accuracy did not improve from 0.09436
663/663 ██████████ 455s 686ms/step - accuracy: 0.1388 - loss: 2.0000
Epoch 11/50
663/663 ██████████ 0s 596ms/step - accuracy: 0.1496 - loss: 2.0000
Epoch 11: val_accuracy did not improve from 0.09436
663/663 ██████████ 461s 695ms/step - accuracy: 0.1457 - loss: 2.0000
Epoch 11: early stopping
Restoring model weights from the end of the best epoch: 1.
Phase 2 best val_accuracy: 9.44 %

```

▼ Step 12 — Evaluation

```

from sklearn.metrics import classification_report, confusion_matrix

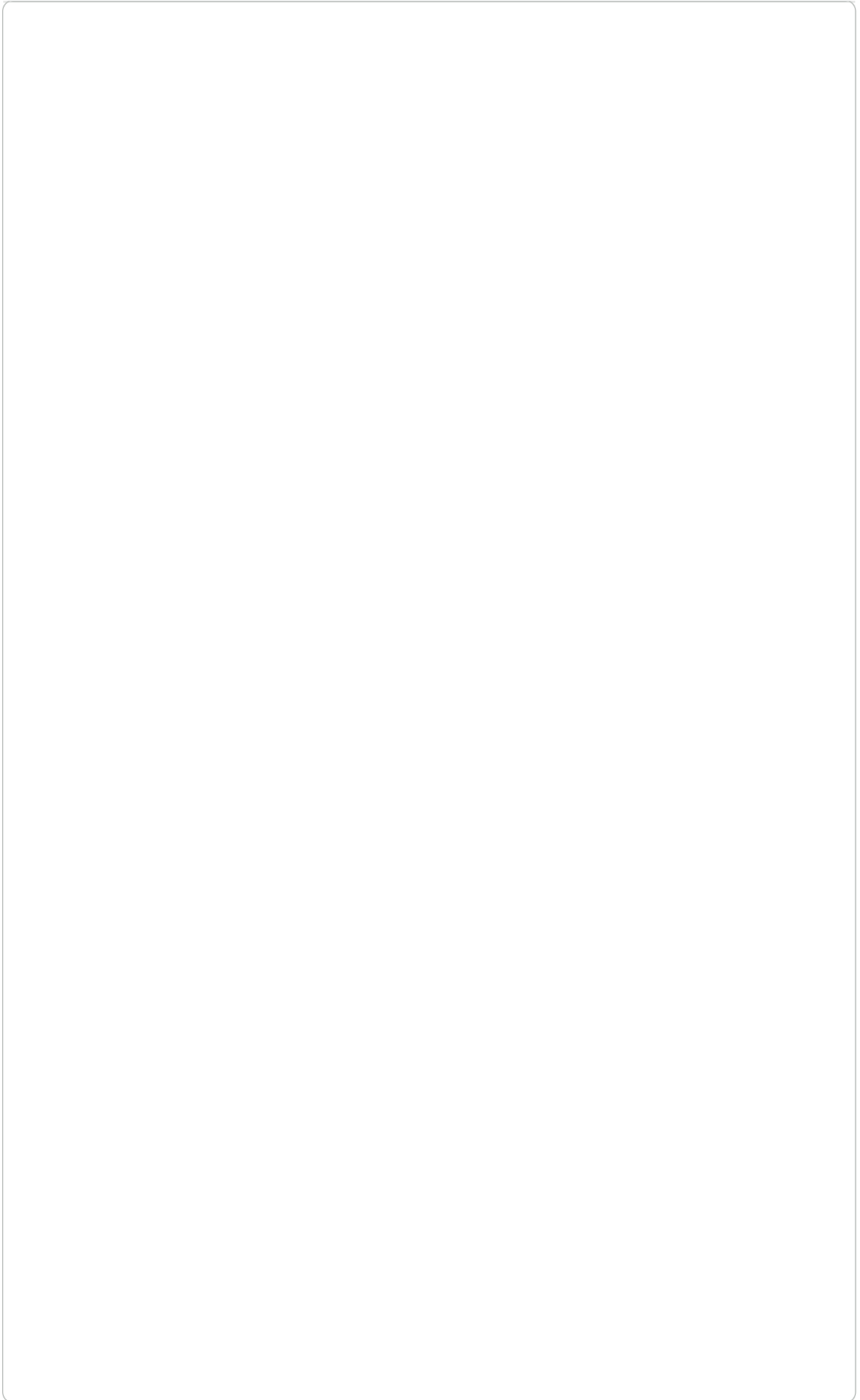
val_gen.reset()
y_true = val_gen.classes
y_prob = model.predict(val_gen, verbose=1)
y_pred = np.argmax(y_prob, axis=1)
class_names = [idx_to_class[i] for i in range(NUM_CLASSES)]

print('Classification Report:')
print(classification_report(y_true, y_pred, target_names=class_names,

fig, ax = plt.subplots(figsize=(14, 11))
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names, linewidth

```

```
ax.set_title('VisionCare AI - Confusion Matrix', fontsize=14, fontwei  
ax.set_xlabel('Predicted'); ax.set_ylabel('True')  
plt.xticks(rotation=35, ha='right', fontsize=8)  
plt.tight_layout()  
plt.savefig(OUTPUT_DIR / 'confusion_matrix.png', dpi=150, bbox_inches=  
plt.show()
```



166/166 — 80s 430ms/step

Classification Report:

	precision	recall	f1-score	support
Central Serous Chorioretinopathy	0.00	0.00	0.00	5
Diabetic Retinopathy	0.00	0.00	0.00	6
Disc Edema	0.00	0.00	0.00	5
Glaucoma	0.00	0.00	0.00	5
Healthy	0.00	0.00	0.00	5
Macular Scar	0.00	0.00	0.00	5
Myopia	0.00	0.00	0.00	5
Pterygium	0.09	1.00	0.17	5
Retinal Detachment	0.00	0.00	0.00	5
Retinitis Pigmentosa	0.00	0.00	0.00	5

Step 13 — Training History

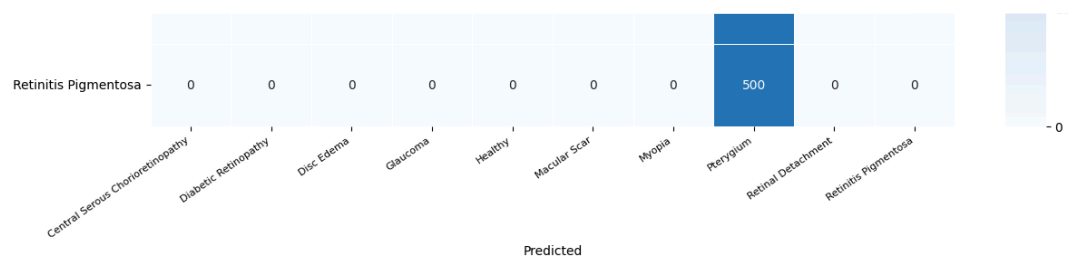
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

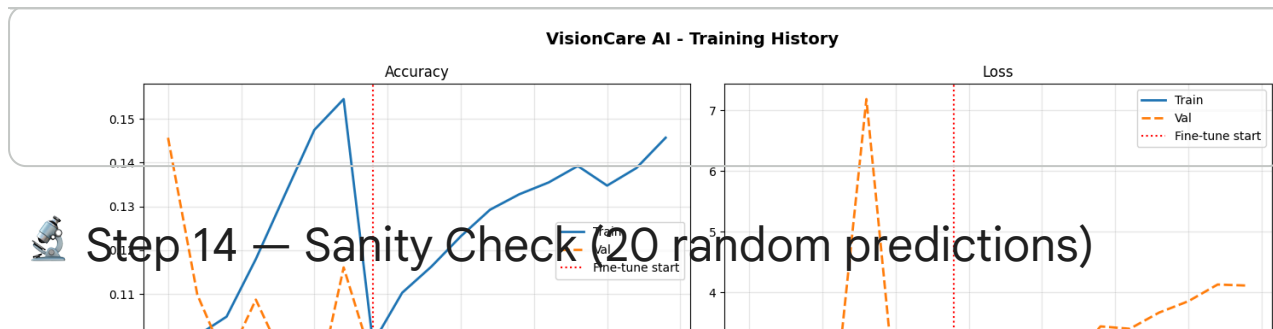
all_acc = history1.history['accuracy'] + history2.history['accuracy']
all_val = history1.history['val_accuracy'] + history2.history['val_accuracy']
all_loss = history1.history['loss'] + history2.history['loss']
all_vloss = history1.history['val_loss'] + history2.history['val_loss']
split = len(history1.history['accuracy'])

ax1.plot(all_acc, label='Train', linewidth=2)
ax1.plot(all_val, label='Val', linewidth=2, linestyle='--')
ax1.axvline(x=split, color='red', linestyle=':', label='Fine-tune start')
ax1.set_title('Accuracy'); ax1.legend(); ax1.grid(alpha=0.3)

ax2.plot(all_loss, label='Train', linewidth=2)
ax2.plot(all_vloss, label='Val', linewidth=2, linestyle='--')
ax2.axvline(x=split, color='red', linestyle=':', label='Fine-tune start')
ax2.set_title('Loss'); ax2.legend(); ax2.grid(alpha=0.3)

plt.suptitle('VisionCare AI - Training History', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(OUTPUT_DIR / 'training_history.png', dpi=150, bbox_inches='tight')
plt.show()
```





Step 14 — Sanity Check (20 random predictions)

```
print('Testing on 20 random images:')
print('-' * 80)
preds = []
for _ in range(20):
    cls = random.choice(class_names)
    imgs = list((WORK_DIR / cls).glob('*.jpg'))
    if not imgs: continue
    img_path = random.choice(imgs)
    img = PILImage.open(img_path).resize((IMG_SIZE, IMG_SIZE)).convert('RGB')
    arr = np.expand_dims(np.array(img) / 255.0, axis=0)
    probs = model.predict(arr, verbose=0)[0]
    pred_cls = idx_to_class[np.argmax(probs)]
    conf = probs[np.argmax(probs)]
    ok = 'OK' if pred_cls == cls else 'WRONG'
    print('  [{}] True: {:<35} Pred: {:<35} ( {:.1%} )'.format(ok, cls, pred_cls, conf))
    preds.append(pred_cls)

print('\nPrediction spread (should NOT be all one class):')
for cls, count in Counter(preds).most_common():
    print('  {:<45} {}'.format(cls, count))
```

Testing on 20 random images:

```
-----
[WRONG] True: Macular Scar                      Pred: Pterygium
[OK] True: Pterygium                            Pred: Pterygium
[OK] True: Pterygium                            Pred: Pterygium
[WRONG] True: Retinitis Pigmentosa              Pred: Pterygium
```